

Datamanagement & Analytics

Prof. Dr. André Drews

Fachbereich Maschinenbau und Wirtschaft

30. März 2026

- nur für den Gebrauch innerhalb des Kurses -

Vorlesungskapitel 2

- Daten, Datentypen, Datenformate -

Lernziele

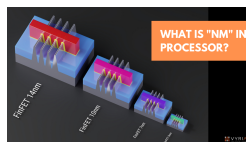
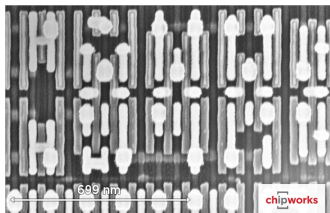
- Einordnung Daten
- Verständnis und Anwendungskompetenz Datenformate
- Einblick in Datenklassifikation

1. Grundlagen Daten
2. Daten im Firmenkontext
3. Filesysteme
4. Datenströme und Prozessmanagement

1. **Kundeninformationen:** Name, Adresse, Kontaktdaten, Kaufhistorie, Vorlieben, Beschwerden usw.
2. **Produktdaten:** Artikelnummer, Beschreibung, Preis, Lagerbestand, Lieferanteninformationen, Produktionsdetails usw.
3. **Finanzdaten:** Umsatz, Gewinn, Verlust, Bilanz, Cashflow, Budgets, Rechnungen, Zahlungen, Kreditoren- und Debitorendaten usw.
4. **Personaldaten:** Mitarbeiterinformationen, wie Name, Adresse, Kontaktdaten, Gehalt, Position, Arbeitszeit, Leistungsbewertungen, Urlaubsansprüche usw.
5. **Lieferantendaten:** Informationen zu Lieferanten, wie Name, Adresse, Kontaktdaten, Verträge, Lieferhistorie, Zahlungsbedingungen usw.

6. **Bestelldaten:** Bestellhistorie, Bestellmenge, Lieferzeit, Bestellstatus usw.
7. **Projektdaten:** Projektbeschreibungen, Meilensteine, Zeitpläne, Ressourcenzuweisungen, Budgets, Fortschrittsberichte usw.
8. **Compliance-Daten:** Gesetze, Vorschriften, Zertifizierungen, Genehmigungen, Datenschutzinformationen, Sicherheitsrichtlinien usw.
9. **Marketingdaten:** Marketingkampagnen, Zielgruppenanalysen, Klickraten, Conversion-Rates, Social-Media-Daten usw.
10. **Betriebsdaten:** Maschinenauslastung, Energieverbrauch, Produktionsdurchsatz, Qualitätssicherungsdaten, Wartungsprotokolle usw.

- Digitalelektronik (digis von griechisch "Finger" - Elektronik der diskreten elektrischen Zustände) eignet sich besonders gut für Datenverarbeitung, da sie am kontrollierbarsten ist. Bauelemente können u.a. im nm-Bereich gefertigt werden.

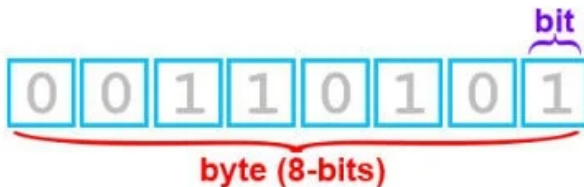


- Binärlogik ("1", "0") ist die einfachste Form der Digitalelektronik

→ deshalb hat sich Digitalisierung durchgesetzt

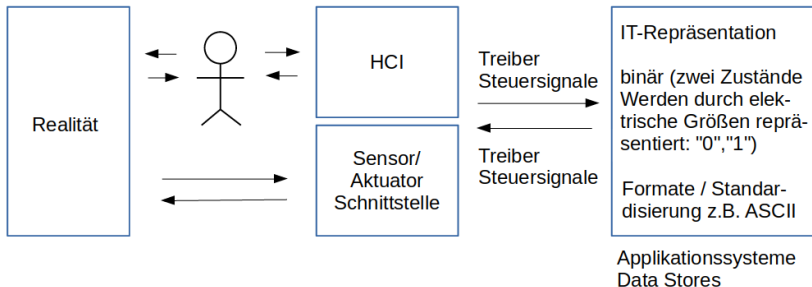
Quelle: <https://www.extremetech.com/computing/193200-intels-14nm-broadwell-chip-reverse-engineered-reveals-impressive-finfets-13-layer-design>

Bits und Bytes

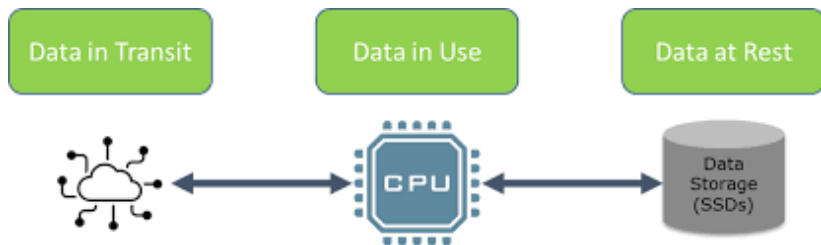


Quelle: <https://www.colocationamerica.com/blog/difference-between-bits-and-bytes>

Digitale Repräsentation von Daten



Data at rest, at transit, in use



Datenrepräsentation über Datentypen

Datentypen			
Gruppe	Typ	Wertebereich	Speicherplatz
Ganze Zahlen	byte	0 bis +255	1 Byte
	short	-32.768 bis +32.767	2 Byte
	int ★	-2.147.483.648 bis +2.147.483.647	4 Byte
	long	-9.223.372.036.854.775.808 bis +9.223.372.036.854.775.80747	8 Byte
Dezimalzahlen	float	Einfache Genauigkeit (7 Nachkommastellen)	4 Byte
	double ★	Einfache Genauigkeit (16 Nachkommastellen)	8 Byte
Sonstiges	char	'A', 'a', 'C' usw. (Einzelne Buchstaben)	2 Byte
	boolean	true oder false	1 Byte
Komplex	string ★	Zeichenkette (Text)	

: Quelle <https://editechlab.com/datentypen-einfach-erklart/?v=3a52f3c22ed6>

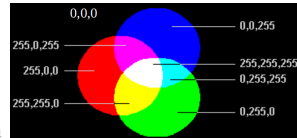
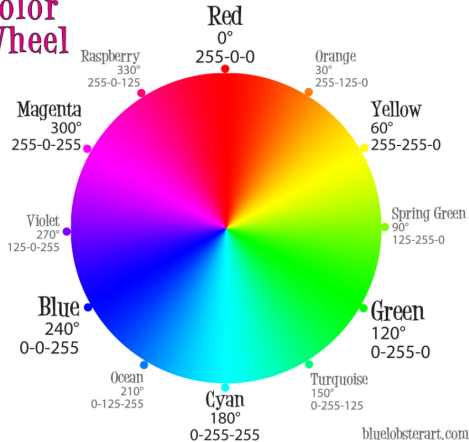
Datenrepräsentation über Datentypen

	ASCII Art	Deutsch					Ressourcen
58	072	3A	00111010	:	:	:	Doppelpunkt
59	073	3B	00111011	;	;	;	Semikolon (Strichpunkt)
60	074	3C	00111100	<	<	<	Kleiner-als-Zeichen
61	075	3D	00111101	=	=	=	Gleichheitszeichen
62	076	3E	00111110	>	>	>	Größer-als-Zeichen
63	077	3F	00111111	?	?	?	Fragezeichen
64	100	40	01000000	@	@	@	at-Zeichen
65	101	41	01000001	A	A		Großbuchstabe A
66	102	42	01000010	B	B		Großbuchstabe B
67	103	43	01000011	C	C		Großbuchstabe C
68	104	44	01000100	D	D		Großbuchstabe D
69	105	45	01000101	E	E		Großbuchstabe E
70	106	46	01000110	F	F		Großbuchstabe F
71	107	47	01000111	G	G		Großbuchstabe G
72	110	48	01001000	H	H		Großbuchstabe H

: Quelle <https://www.ascii-code.com/de>

Datenrepräsentation über Datentypen

RGB Color Wheel

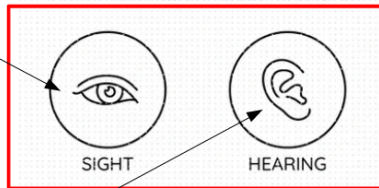


: Quelle: <https://webriti.com/wp-content/uploads/2012/01/rgb-color-wheel-lg.jpg>

HCI: Display:
Textdaten, Bilder,
Bewegtbilder/Videos,
...
(Datenformat: .txt,
.png, .mp4,...)

HCI: Lautsprecher, Mikrophon
Audiodaten
(Datenformat: .mp3, ...)

Kommunikation/ Datentransfer



© Copyright GHEMGEESVG
ghemgeesvg.com

Datenformat

Standardisierung, welche die Struktur vorgibt wie unterschiedliche Daten einem Gesamtkontext zugeordnet werden können.

- Datenformate erleichtern die Verarbeitung von Daten über Informationstechnologie.
- Es existieren Universalformate (.xml, .json, ...) und Applikationsspezifische Formate (vornehmlich binär wie .pptx, .docx, ...).

Universalformate:

- Text: .txt, .csv, .xml/.xsd, .json, .yaml, ...
- Bild: .bmp, .png, .jpg, .svg, ...
- Video: .avi, .mpeg4, ...
- Audio: .wav, .mp3, ...
- Dokumente: .pdf, .dvi, .eps, ...
- Industrie: OPC, ...

Anwendungsspezifische Formate:

- Microsoft Office: .docx, .xlsx, .pptx, Libre Office: odp, .calc.
- Power BI: .pbix, ...

Unterstützte Datenformate von PowerBI

Daten abrufen

Suchen

Alle

Datei

Datenbank

Microsoft Fabric

Power Platform

Azure

Onlinedienste

Sonstige

Datei

 Excel-Arbeitsmappe

 Text/CSV

 XML

 JSON

 Ordner

 PDF

 Parquet

 SharePoint-Ordner

Kunde:

Name: Max Mustermann

Adresse: Musterstraße 123

Stadt: Musterstadt

Produkte:

1. LKW-Reifen - Menge: 4

2. LKW-Batterie - Menge: 2

```
def read_txt_file(file_path):  
    try:  
        with open(file_path, 'r') as file:  
            content = file.read()  
            return content  
    except FileNotFoundError:  
        print("Die Datei wurde nicht gefunden.")  
        return None
```

.csv (comma seperated values)

```
Bestellnummer,Lieferdatum,Name,Adresse,Stadt,Produkt,Menge  
123456,2024-03-01,Max Mustermann,Musterstraße 123,Musterstadt,LKW-Reifen,4  
123456,2024-03-01,Max Mustermann,Musterstraße 123,Musterstadt,LKW-Batterie,2
```

```
import csv  
  
def read_csv_file(file_path):  
    try:  
        with open(file_path, newline='') as csvfile:  
            reader = csv.reader(csvfile)  
            for row in reader:  
                print(row)  
    except FileNotFoundError:  
        print("Die Datei wurde nicht gefunden.")
```

.xml (Extendable Markup Language)

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <person>
    <name>Max</name>
    <age>30</age>
    <city>Berlin</city>
  </person>
  <person>
    <name>Anna</name>
    <age>25</age>
    <city>München</city>
  </person>
  <person>
    <name>Jonas</name>
    <age>35</age>
    <city>Hamburg</city>
  </person>
</root>
```

```
import xml.etree.ElementTree as ET

# Parse the XML file
tree = ET.parse('example.xml')
root = tree.getroot()

# Iterate through the XML tree
for child in root:
    print(child.tag, child.attrib)

# Access specific elements
for elem in root.iter('specific_element'):
    print(elem.tag, elem.text)
```

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://example.com/logistik"
  xmlns:logistik="http://example.com/logistik"
  <xs:element name="bestellung">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="bestellnummer" type="xs:integer"/>
        <xs:element name="lieferdatum" type="xs:date"/>
        <xs:element name="lieferadresse">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="adresse" type="xs:string"/>
              <xs:element name="stadt" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
        <xs:element name="produkt">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="menge" type="xs:integer"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<logistik:bestellung xmlns:logistik="http://example.com/logistik">
  <logistik:bestellnummer>123456</logistik:bestellnummer>
  <logistik:lieferdatum>2024-03-01</logistik:lieferdatum>
  <logistik:lieferadresse>
    <logistik:name>Max Mustermann</logistik:name>
    <logistik:adresse>Musterstraße 123</logistik:adresse>
    <logistik:stadt>Musterstadt</logistik:stadt>
  </logistik:lieferadresse>
  <logistik:produkt>
    <logistik:name>LKW-Reifen</logistik:name>
    <logistik:menge>4</logistik:menge>
  </logistik:produkt>
</logistik:bestellung>
```

```
from lxml import etree

# Load XML file
xml_file = 'example.xml'
with open(xml_file, 'rb') as f:
    xml_data = f.read()

# Load XSD schema
xsd_file = 'schema.xsd'
with open(xsd_file, 'rb') as f:
    xsd_data = f.read()

# Parse XSD schema
schema = etree.XMLSchema(etree.fromstring(xsd_data))

# Parse XML file
parser = etree.XMLParser(schema=schema)
try:
    tree = etree.parse(xml_file, parser)
    print("XML is valid")
except etree.XMLSyntaxError as e:
    print("XML is invalid:", e)
```

.json (Java Script Object Notation)

```
{
  "bestellnummer": 123456,
  "lieferdatum": "2024-03-01",
  "lieferadresse": {
    "name": "Max Mustermann",
    "adresse": "Musterstraße
123",
    "stadt": "Musterstadt"
  },
  "produkte": [
    {
      "name": "LKW-Reifen",
      "menge": 4
    },
    {
      "name": "LKW-Batterie",
      "menge": 2
    }
  ]
}
```

```
import json

# Open the JSON file
with open('data.json', 'r') as file:
    # Load JSON data
    data = json.load(file)

# Now 'data' contains the contents of the JSON file as a Python dictionary
print(data)
```



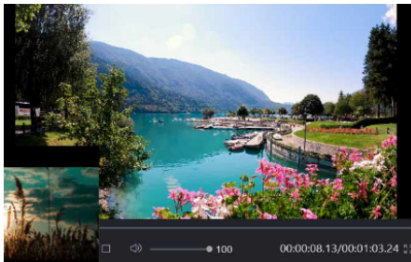
```
from PIL import Image

# Bild einlesen
image = Image.open("beispiel.png")

# Informationen über das Bild anzeigen
print("Bildformat:", image.format)
print("Bildgröße:", image.size)
print("Modus:", image.mode)

# Bild anzeigen
image.show()
```

.avi file



```
video_path = 'beispiel.avi'

# Video-Objekt erstellen
video = cv2.VideoCapture(video_path)

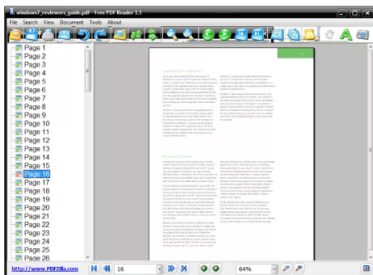
# Schleife zum Lesen und Anzeigen der Frames
while video.isOpened():
    # Frame lesen
    ret, frame = video.read()

    # Überprüfen, ob das Video zu Ende ist
    if not ret:
        break

    # Frame anzeigen
    cv2.imshow('Video', frame)

    # Warten auf Tastendruck (25 FPS)
    if cv2.waitKey(40) & 0xFF == ord('q'):
        break

# Video-Objekt und Fenster schließen
video.release()
cv2.destroyAllWindows()
```



```
import PyPDF2

# PDF-Datei öffnen
with open('beispiel.pdf', 'rb') as file:
    # PDF-Reader erstellen
    reader = PyPDF2.PdfFileReader(file)

    # Anzahl der Seiten im PDF ermitteln
    num_pages = reader.numPages

    # Schleife zum Lesen des Inhalts jeder Seite
    for page_num in range(num_pages):
        # Seite lesen
        page = reader.getPage(page_num)

        # Text extrahieren
        text = page.extractText()

        # Text anzeigen
        print("Inhalt der Seite", page_num + 1, ":", text)
```

.wav file



```
import wave

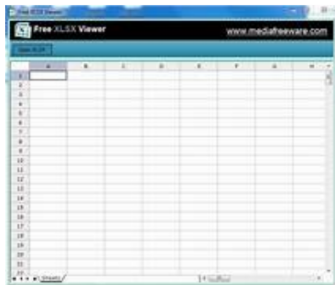
# Replace 'audio.wav' with the path to your WAV file
file_path = 'audio.wav'

# Open the WAV file
with wave.open(file_path, 'rb') as wave_file:
    # Get basic information about the WAV file
    num_channels = wave_file.getnchannels()
    sample_width = wave_file.getsampwidth()
    frame_rate = wave_file.getframerate()
    num_frames = wave_file.getnframes()

    # Read all the frames from the WAV file
    frames = wave_file.readframes(num_frames)

# Print basic information about the WAV file
print(f"Number of channels: {num_channels}")
print(f"Sample width (bytes): {sample_width}")
print(f"Frame rate (frames per second): {frame_rate}")
print(f"Number of frames: {num_frames}")
```

.xlsx file



```
import pandas as pd

# Replace 'file.xlsx' with the path to your Excel file
file_path = 'file.xlsx'

# Read the Excel file into a pandas DataFrame
df = pd.read_excel(file_path)

# Now you can work with your data stored in the DataFrame 'df'
print(df)
```

1. Grundbegriffe
2. Grundlagen Daten
3. Daten im Firmenkontext
4. Filesysteme
5. Datenströme und Prozessmanagement

Firmenname

Firmenname – Musterstraße 51 – 12345Stadt

Mustermann GmbH
Herrn Max Mustermann
12345 Stadthausen

Firmenname
Musterstraße 51
12345 Stadt

Tel.: 0211 12345 67
E-Mail: info@domain.de
Internet: www.domain.de

Lieferschein

LieferNr.: BestellNr.: Kunden-Nr.: 1003 Datum: 07.08.2012

Vielen Dank für Ihre Bestellung. Wir liefern Ihnen wie vereinbart folgende Waren:

Art.-Nr.	Bezeichnung	Anzahl	Einzelpreis	Gesamtpreis
11111	Dienstleistung	1 Stunde	60,00 EUR	60,00 EUR
22222	Artikel	2 Stück	5,00 EUR	10,00 EUR
Zwischensumme				70,00 EUR
19% MwSt.				13,30 EUR
Gesamtbetrag				83,30 EUR

Die gelieferte Ware bleibt bis zur vollständigen Bezahlung unser Eigentum.

Kostenlose Rechnungsvorlage von <http://rechnungen-muster.de>.

- nur für den Gebrauch innerhalb der Vorlesung -

Daten im Flottenmanagement der Transportlogistik

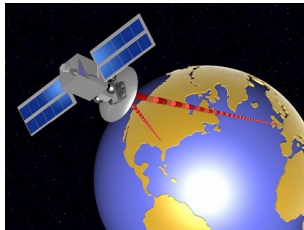
[illegible]

Quelle: <https://www.soloplan.de/produkte/carlo/#Transportmanagementsystem>, Oktober 2020

GPS-Daten gemäß Koordinatensystem World Geodetic System 1984 (WGS84)

Darstellung	Beispiel	Beschreibung
hddd° mm' ss.ss"	N46°14'06.70" E008°00'55.60"	Angabe in Graden(°), Minuten('), Sekunden(") und Dezimalsekunden
hddd° mm.mmm'	N46°14.11182' E008°00.92670'	Angabe in Graden(°), Minuten(') und Dezimalminuten
hddd.ddddd°	N46.235197° E008.015445°	Angabe in Graden(°) und Dezimalgraden
±ddd.ddddd°	Lat=46.235197° Long=008.015445°	Angabe in ±Graden(°) und Dezimalgraden

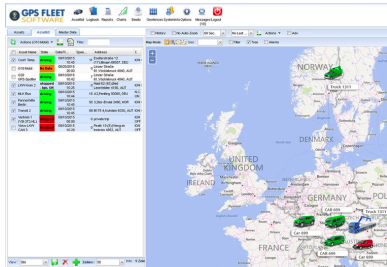
Quelle: www.wikipedia.org, Oktober 2019



Quelle: <https://logistikknowhow.com/materialfluss-und-transport/global-positioning-system-gps/>, Oktober 2019

Real Time Location Systems (RTLS)

- Real Time Location Systems (RTLS): Systeme, um Objekte und Personen in nahezu Echtzeit zu identifizieren und lokalisieren.



Quelle: www.solconica.de, Oktober 2019, www.gps.at, Oktober 2019

Daten in der Intralogistik (z.B. Warenannahme) - Barcode

- Bei Barcodes handelt es sich um eine optoelektronisch lesbare Schrift, die aus verschiedenen breiten, parallelen Strichen und Lücken besteht.
- Die Striche können mit einem optischen Lesegerät (Barcode-Scanner) maschinell eingelesen und anschließend elektronisch verarbeitet werden.
- Das Wort "Code" bezieht sich auf Codierung von Zeichen und Buchstaben in Striche und Lücken.



<https://www.activebarcode.de/codes/eanucc128.html>

<https://www.bluelinkerp.com/blog/2019/03/14/optimize-inventory-management-with-wireless-barcode-scanning/>

Name	Beschreibung	Beispiel
GS1	• der Zeichensatz besteht aus 74 numerische, 41 alphanumerische Zeichen • der Code besteht aus 7 möglichen Strichcodevarianten • Anders als beim EAN-Barcode werden Zusatzinformationen wie Gewicht und Mindesthaltbarkeitsdatum mitgegeben. • GS1 wird bevorzugt bei Lebensmitteln (Point of Sale (Supermarkt etc.)) eingesetzt.	

<https://www.activebarcode.de/codes/eanucc128.html>, Oktober 2019

Prüfziffer

Bei der Übermittlung von Daten wird aus den Rohdaten mit einer festgelegten Berechnungsvorschrift eine Prüfziffer generiert. Diese dient dazu Eingabefehler bzw. Übertragungsfehler zu detektieren.

Prüfziffernberechnung EAN 8

Die Berechnung der Prüfziffer erfolgt nach **Modulo 10** mit der **Gewichtung 3**. Bei der Berechnung wird mit der ersten Nutzziffer von rechts mit dem Gewichtungsfaktor 3 begonnen.

Beispiel:



Klartext:	4 0 1 2 3 4 5 5
Ziffernstelle:	8 7 6 5 4 3 2 1
Prüfziffer:	5
Nutzziffernfolge:	4 0 1 2 3 4 5
Gewichtungsfaktoren:	3 1 3 1 3 1 3
Einzelprodukte:	12 0 3 2 9 4 15
Summe der Einzelprodukte:	$12 + 0 + 3 + 2 + 9 + 4 + 15 = 45$
Modulo 10:	$45 \text{ Mod. } 10 = 5 \quad (45 / 10 = 4 \text{ Rest } 5)$
Differenz zum nächsten Vielfachen von 10 ergibt die Prüfziffer:	$10 - 5 = 5$
Prüfziffer:	5

Quelle: http://www.kbconsult.ch/PdF%20File/Barcode_Pruefzifferberechnung.pdf

Prüfziffernberechnung Code 39

Die Prüfziffer wird nach **Modulo 43** berechnet.

Jedem Zeichen vom Code 39 ist eine Zahl zugeordnet. Diese werden für die Prüfzifferberechnung aufaddiert und durch 43 geteilt. Dem Rest aus der Division wird wieder ein Zeichen aus der Referenztabelle zugeordnet.

Beispiel:



Klartext:	KBCONSULT
Prüfziffer (-zeichen):	Q
Nutzzeichenfolge:	KBCONSULT
Summe der Referenzzahlen:	$20+11+12+24+23+28+30+21+29 = 198$
Modulo 43:	$198 \text{ Mod. } 43 = 27$ (199/43 = 4 Rest 26)
Prüfziffer:	26
Darstellbares Zeichen:	Q

Referenztabelle:

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
W	X	Y	Z	-	.	Space	\$	/	+	%					
32	33	34	35	36	37	38	39	40	41	42					

Quelle: http://www.kbconsult.ch/PdF%20File/Barcode_Pruefzifferberechnung.pdf

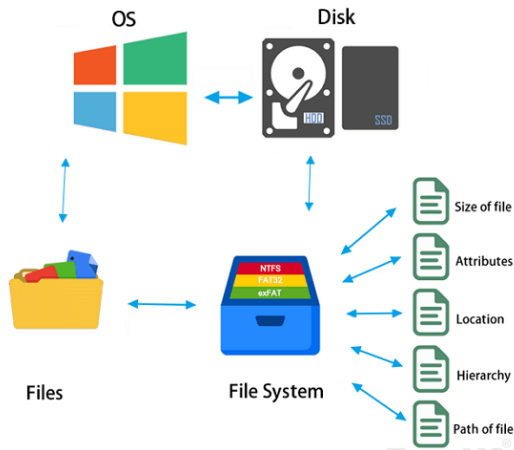
Table of Contents

1. Grundbegriffe
2. Grundlagen Daten
3. Daten im Firmenkontext
4. Filesysteme
5. Datenströme und Prozessmanagement

Dateien

Zusammenfassung gleichartiger digitaler Daten, die zum Speichern auf Datenträgern oder Speichermedien, zur Wiedergabe, zum Bearbeiten und zur Datenübertragung dient und durch einen Dateinamen identifiziert wird.

- Daten lassen sich aus dem Hauptspeicher heraus (data at process) als Dateien auf z.B. Festplatten Persistieren (data at rest). Dort müssen sie verwaltet werden.
- Eine einfach Form des Datenmanagementsystems (speichern, organisieren und verwalten von Dateien) zwischen Betriebssystem und Persistenz von Dateien auf Festplatte ist das file/Dateisystem



Quelle: <https://www.easeus.de/festplattenverwaltung/dateisystem.html>

Eigenschaften von Filesystemen

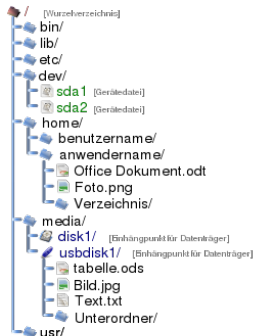
Dateisystem	Max. Dateigröße	Max. Volumen	Journaling	Snapshots	Typische Nutzung	Wann nutzen?
<u>FAT32</u>	4 GB	2–32 GB	✗	✗	USB, alte Geräte	Maximale Kompatibilität
<u>exFAT</u>	16 EB	128 PB	✗	✗	USB, SD-Karten	Große Dateien + plattformübergreifend
<u>NTFS</u>	~16 TB+	16 EB	✓	✗	Windows-PCs	Standard für Windows
<u>ReFS</u>	35 PB	35 PB+	✓	✗	Server, Storage	Datensicherheit (Windows Server)
<u>ext4</u>	16 TB	1 EB	✓	✗	Linux-Systeme	Stabil, zuverlässig
<u>Btrfs</u>	16 EB	16 EB	✓	✓	Linux, NAS	Snapshots, moderne Features
<u>XFS</u>	8 EB	8 EB	✓	✗	Server, große Daten	Sehr hohe Performance
<u>APFS</u>	8 EB	8 EB	✓	✓	macOS	SSD + Apple-Ökosystem
<u>ZFS</u>	16 EB	256 ZB	✓	✓	NAS, Server	Maximale Datensicherheit

- **Journaling:** Eine Technik, bei der Änderungen zuerst in ein Protokoll geschrieben werden, um das Dateisystem nach Abstürzen schnell und sicher wiederherstellen zu können.
- **Snapshots:** Eine Funktion, die eine Momentaufnahme des Dateisystems speichert, sodass man jederzeit zu einem früheren Zustand zurückkehren kann.

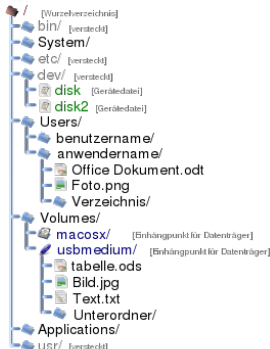
- nur für den Gebrauch innerhalb der Vorlesung -

Filesystem Betriebssystem abhängige Baumstruktur

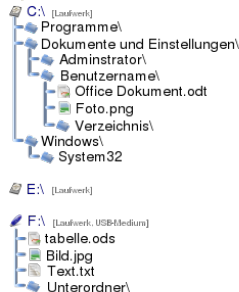
Linux, unixolde, ZETA



Mac OSX

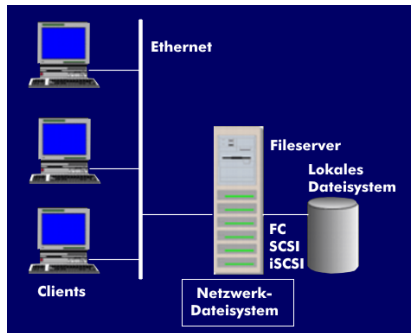


Windows NT/2000/XP



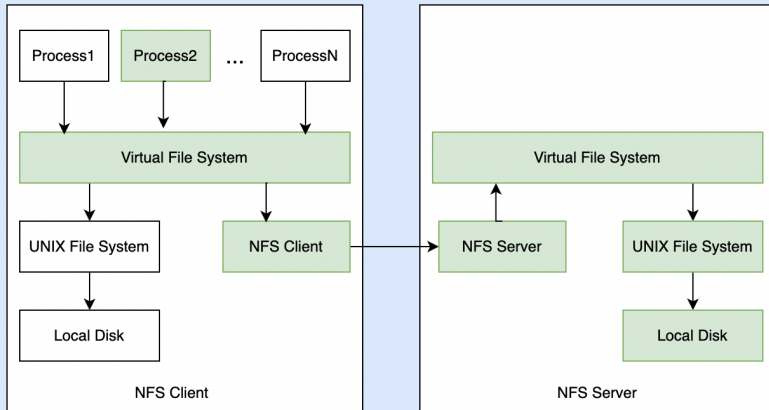
Quelle: <https://de.wikipedia.org/wiki/Dateisystem>

Netzwerk- Dateisystem ist ein verteiltes Dateisystem. Es dient dem Filesharing und bietet Anwendern und Anwendungen Zugriff auf Daten und Dateien, die auf entfernten Rechnern liegen. Diese können so bearbeitet werden, als lägen sie auf dem lokalen Rechner
→ file cloud



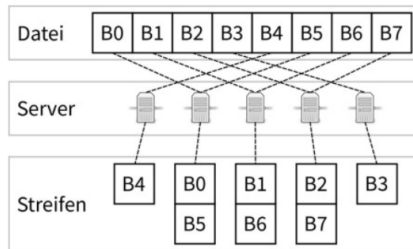
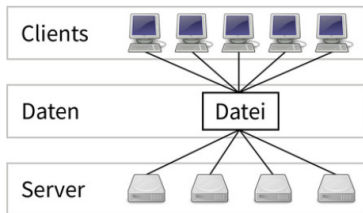
Quelle: <https://www.itwissen.info/Netzwerk-Dateisystem-network-file-system.html>

NFS Architecture



Quelle: <https://mohitdtumce.medium.com/network-file-system-nfs-d0c08e191ab2>

Parallele Filesysteme (z.B. HDFS, Big Data)



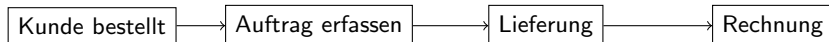
Quelle: <https://gi.de/informatiklexikon/parallele-dateisysteme>

1. Grundbegriffe
2. Grundlagen Daten
3. Daten im Firmenkontext
4. Filesysteme
5. Datenströme und Prozessmanagement

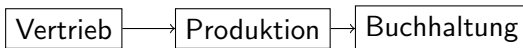
- nur für den Gebrauch innerhalb der Vorlesung -



Datenfluss entlang eines Prozesses



- Daten werden entlang des Prozesses verändert und/oder angereichert
- Jeder Schritt erzeugt neue Datentypen



- Daten werden mehrfach gespeichert
- Medienbrüche entstehen
- Risiken:
 - ▶ Inkonsistenz
 - ▶ Verzögerungen

- Ziel: **Durchgängiger Datenfluss**
- Eigenschaften:
 - ▶ Zentrale Datenhaltung
 - ▶ Einheitliche Datenmodelle
 - ▶ Echtzeit-Zugriff
- Beispiel:
 - ▶ ERP-System als zentrale Datenquelle

- Die Organisationsstruktur gibt Prozesse vor
- Prozesse arbeiten auf Daten und generieren Datenströme
- Datenmanagement bildet die Voraussetzung für Datenströme
- Zusammenspiel entscheidet über:
 - ▶ Effizienz
 - ▶ Datenqualität
 - ▶ Analysefähigkeit

- Prozesse verlaufen oft über mehrere Abteilungen
- Organisationsstruktur bestimmt:
 - ▶ Schnittstellen zwischen Daten
 - ▶ Übergabepunkte
- Problem:
 - ▶ Jede Abteilung verwaltet eigene Daten



Startups, Kleinunternehmen, kleine Mittelständler

Organisationsform - Einlinienorganisation - Datenmanagement

- Daten werden entlang klar definierter hierarchischer Linien weitergegeben.
- Jede Organisationseinheit verwaltet nur ihre eigenen Daten.
- Zugriff auf Daten erfolgt über den jeweiligen Vorgesetzten.
- Es gibt keine direkte Datenweitergabe zwischen gleichrangigen Abteilungen.

Beispiel:

- Ein Mitarbeiter erfasst Kundendaten im Vertrieb.
- Diese werden an die Vertriebsleitung weitergegeben.
- Die Vertriebsleitung gibt relevante Daten an die Geschäftsführung weiter.
- Andere Abteilungen (z. B. Buchhaltung) erhalten die Daten nur über diese Hierarchie.



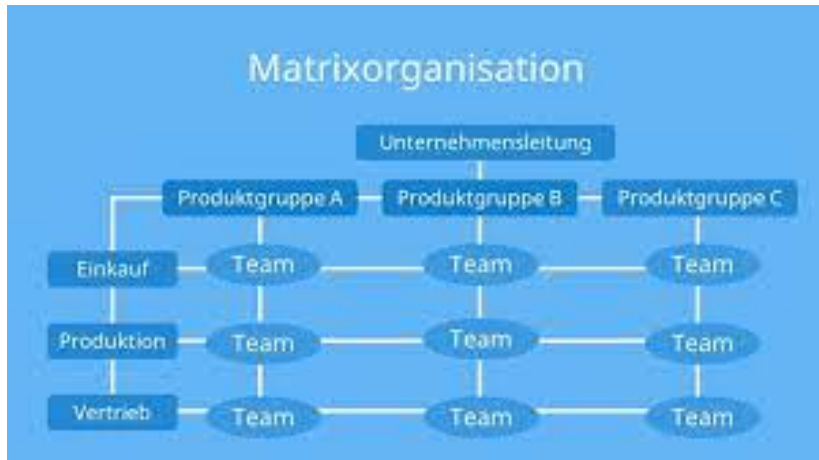
schnell wachsende Unternehmen

Organisationsform - Mehrlinienorganisation - Datenmanagement

- Daten können über mehrere fachliche Linien und Vorgesetzte fließen.
- Organisationseinheiten teilen Daten bereichsübergreifend.
- Zugriff auf Daten erfolgt durch mehrere verantwortliche Stellen.
- Direkte Datenweitergabe zwischen Abteilungen ist möglich und üblich.

Beispiel:

- Ein Mitarbeiter erfasst Kundendaten im Vertrieb.
- Diese Daten stehen gleichzeitig dem Marketing und der Buchhaltung zur Verfügung.
- Fachvorgesetzte aus verschiedenen Bereichen greifen direkt auf die Daten zu.
- Entscheidungen werden gemeinsam auf Basis der geteilten Daten getroffen.



- Konzerne

Organisationsform - Matrixorganisation - Datenmanagement

- Daten fließen sowohl entlang funktionaler als auch projektbezogener Linien.
- Organisationseinheiten teilen Daten funktions- und projektübergreifend.
- Zugriff auf Daten erfolgt durch mehrere Verantwortliche (z. B. Fach- und Projektleitung).
- Hohe Transparenz, aber auch erhöhter Abstimmungsbedarf bei Datenpflege und -nutzung.

Beispiel:

- Ein Mitarbeiter erfasst Kundendaten im Vertrieb.
- Diese Daten werden sowohl vom Abteilungsleiter Vertrieb als auch vom Projektleiter genutzt.
- Das Marketing greift direkt auf relevante Daten für Kampagnen zu.
- Entscheidungen basieren auf gemeinsam genutzten Daten aus verschiedenen Perspektiven.



- Mittlere und große Unternehmen

Organisationsform - Stablinienorganisation - Datenmanagement

- Daten fließen weiterhin entlang der hierarchischen Linien der Linienorganisation.
- Stabsstellen sammeln, analysieren und bereiten Daten für die Entscheidungsfindung auf.
- Stabsstellen haben beratenden Zugriff auf Daten, aber keine Weisungsbefugnis.
- Entscheidungen basieren auf aufbereiteten Informationen der Stabsstellen.

Beispiel:

- Ein Mitarbeiter erfasst Kundendaten im Vertrieb.
- Diese werden an die Vertriebsleitung weitergegeben.
- Eine Stabsstelle (z. B. Controlling) analysiert die Daten und erstellt Berichte.
- Die Geschäftsführung trifft Entscheidungen auf Basis dieser Auswertungen.



Unternehmen mit multifunktionalen Teams

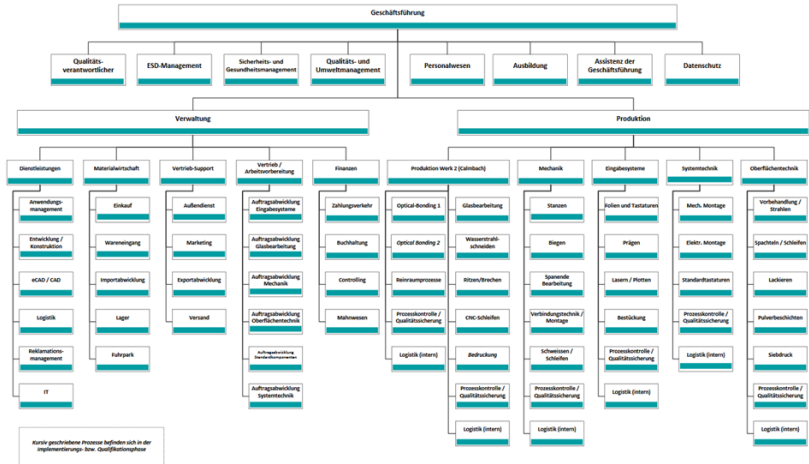
Organisationsform - Netzwerkorganisation - Datenmanagement

- Daten werden dezentral und flexibel zwischen internen und externen Partnern ausgetauscht.
- Organisationseinheiten greifen direkt auf gemeinsame Datenplattformen zu.
- Hoher Grad an Vernetzung ermöglicht schnellen Informationsfluss ohne feste Hierarchien.
- Datenmanagement erfordert klare Standards und Schnittstellen zur Koordination.

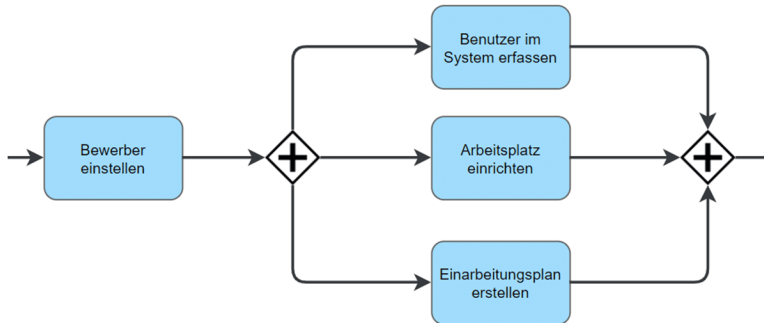
Beispiel:

- Ein Mitarbeiter erfasst Kundendaten im Unternehmen.
- Diese Daten werden in einer gemeinsamen Cloud-Plattform gespeichert.
- Externe Partner (z. B. Logistik oder Marketing) greifen direkt auf relevante Daten zu.
- Entscheidungen werden gemeinsam und zeitnah auf Basis der aktuellen Daten getroffen.

Beispielorganisation

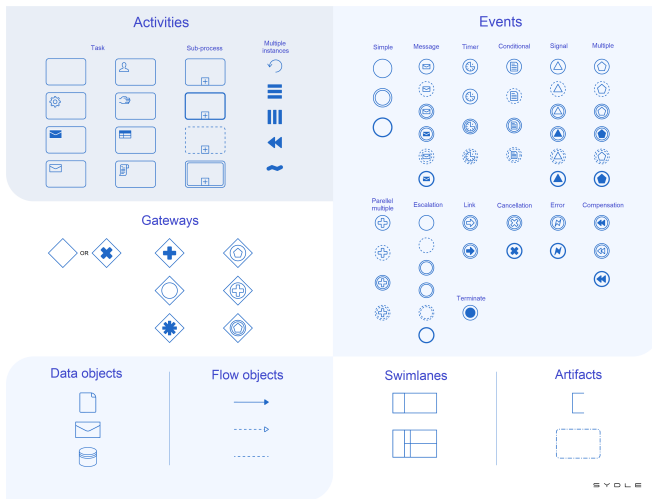


Prozessmodellierung BPMN



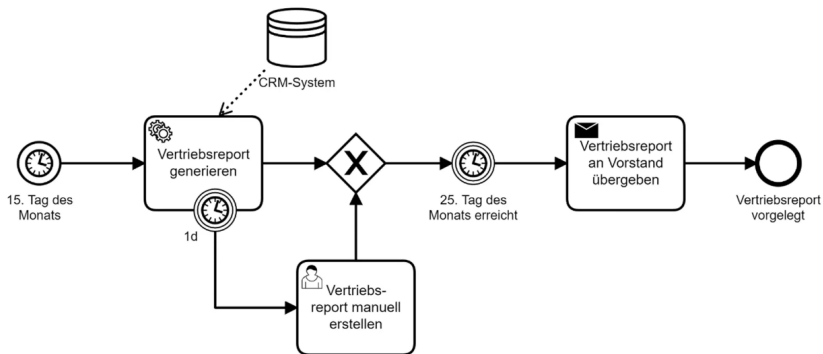
Quelle: <https://www.inveskills.com/bpmn/bpmn-examples/>

Prozessmodellierung BPMN



Quelle: <https://www.inveskills.com/bpmn/bpmn-examples/>

Prozessmodellierung BPMN mit Datenströmen



Quelle: <https://www.roxtra.com/blog/zeitereignisse-einsetzen-bpmn/>

Prozessmodellierung in BPMN mit Datenströmen

